A WORKING GUIDE FOR ENGINEERS NEW TO THE FIELD

Deep Learning and Robotics

The ideas behind today's models and the robots they drive

Matthew Wnuk

15 topics

From the training loop to the robot's body, in the order you meet them on a project.

Ideas over model names

Model names change every few months. The ideas underneath them change slowly.

Meant to be presented

Each slide has one message, a picture, and speaker notes with the full explanation.

OPENING

The map: fifteen topics in four groups

we move from how a model learns, to what models exist, to how they drive robots, to how the work is done

Foundations

1 Things I know

2 Foundations that survived

3 The transformer

Models

4 Representation learning

5 Generative modeling

6 Foundation models and adaptation

Robots

7 Robot learning basics

8 Robot policies today

9 World models

10 Data for robots

Practice

11 Evaluation

12 Training at scale

13 Deployment on the robot

14 Understanding and safety

15 Embodiments

Read left to right: each group builds on the one before it

Six principles that outlast any model

the rest of the deck is detail; these are the habits that decide whether a project works

Data beats clever algorithms

- Scale, diversity and quality of data set the ceiling.
- A better dataset usually helps more than a new architecture.

General methods that scale win

- The "bitter lesson": learning plus compute overtakes hand-built rules.
- Prefer designs that improve with more data.

All models are wrong

- A good offline score is not a working robot.
- Judge the model in the loop it will run in.

Failing models don't throw exceptions

- A broken pipeline still produces a loss curve.
- Look at inputs, labels and outputs yourself.

Evaluation leaks

- Tuning against the test set is training on it.
- Public benchmarks may already be in the training data.

One variable per experiment

- Change one thing, keep a matched control.
- Know the trivial baseline before you start.

When a result surprises you, come back to this slide before changing the model

Silent failure: what it looks like and how to catch it

nothing crashes, the loss goes down, and the model has still learned the wrong thing

Habits that catch it

Look at the data. View raw samples after every transform.

Overfit one batch. If the model cannot memorise 10 samples, there is a bug.

Beat a dumb baseline. Compare with "predict the mean" or "repeat the last action".

Ask what shortcut exists. Could the model score well without solving the task?

Test in the loop. Run the policy, do not only score it.

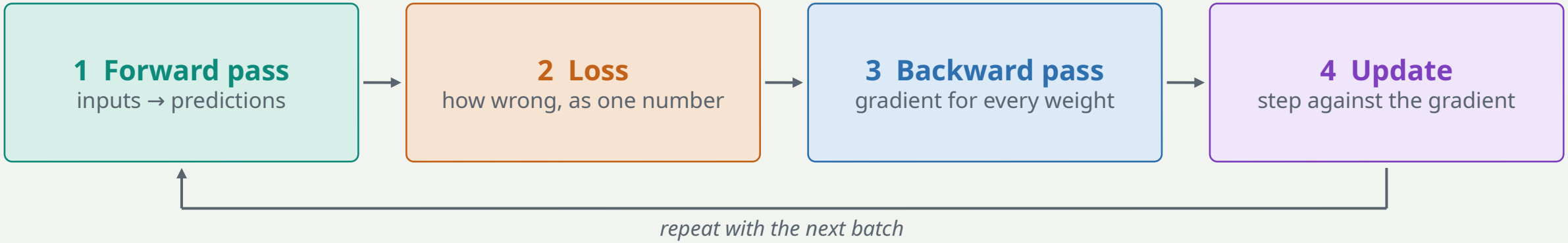
Four common silent failures

Label or input bug	Images and labels misaligned, wrong normalisation, a camera swapped. *Symptom: Loss falls slowly and plateaus high.*
Data leak	Test samples, or near-copies of them, are in the training set. *Symptom: Validation score looks too good.*
Shortcut learning	The model uses a cue that happens to correlate with the answer. *Symptom: Great on the benchmark, fails on new scenes.*
Evaluation bug	The metric is computed on the wrong split or the wrong output. *Symptom: Numbers do not match what you see.*

A falling loss proves the code runs. It does not prove the model learned what you intended

The training loop: four steps, repeated millions of times

every model in this deck, from a small classifier to a robot foundation model, is trained this way



Gradient descent

- The gradient points uphill on the loss.
- We step downhill, a little at a time.
- "Stochastic": each step uses a small batch, not the whole dataset.

Backpropagation

- The chain rule, applied layer by layer from the loss back to the input.
- Frameworks (PyTorch, JAX) do this for you automatically.

AdamW, the default optimizer

- Keeps a running average of each weight's gradient and its size.
- Gives every weight its own step size.
- Weight decay gently pulls weights toward zero.

Training is a search for weights that make the loss small on data the model has not memorised

Learning rate: warm up, then decay

the learning rate is the size of each update step, and it is the most important number you will tune

How to read the schedule

Warmup. Start near zero and ramp up. Early gradients are noisy, and large steps can wreck the model.

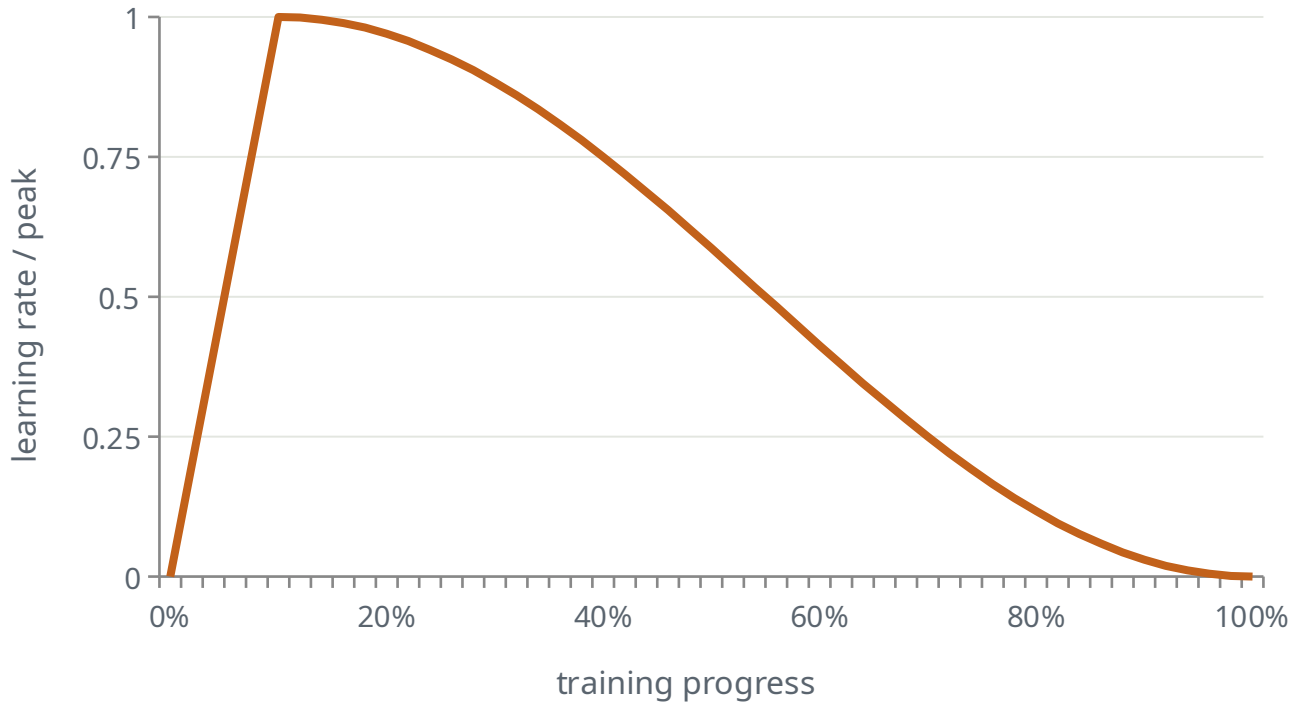
Peak. The highest rate the model can train at without diverging.

Cosine decay. Shrink the step smoothly so the model settles into a good solution.

Too high: loss jumps or becomes NaN.

Too low: loss falls very slowly.

Warmup plus cosine decay (illustrative)



If training diverges or stalls, look at the learning rate before anything else

Loss functions: telling the model what "wrong" means

the loss is the only signal the model gets, so choosing it is choosing what the model cares about

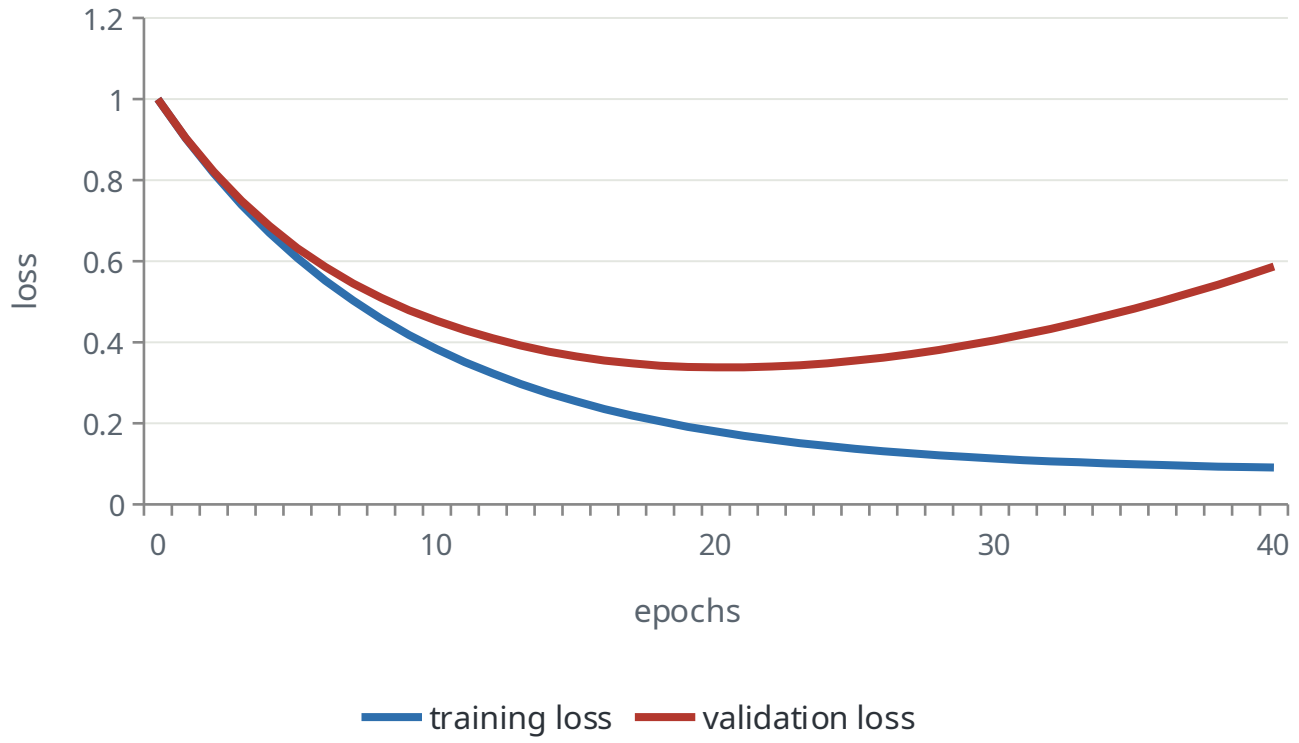
Loss	Use it when the output is	Intuition	Where you will see it
Cross entropy	One of several classes, or the next token	Penalises putting low probability on the correct answer	Classifiers, language models, action tokens
MSE (L2)	A continuous number	Penalises large errors heavily; predicts the average	Regression, diffusion and flow-matching targets
L1 / Huber	A continuous number with outliers	Penalises errors evenly; less swayed by outliers	Action regression, depth, boxes
KL divergence	A probability distribution	How different one distribution is from another	VAEs, distillation from a teacher
Contrastive (InfoNCE)	An embedding	Pull matching pairs together, push the rest apart	CLIP-style models, retrieval, re-identification

Pick the loss that matches the kind of output: a class, a number, a distribution, or a similarity

Overfitting, underfitting and capacity

the goal is low error on data the model has never seen, not on the data it trained on

Training versus validation loss (illustrative)



Terms to know

Capacity: how complex a function the model can represent; roughly, its parameter count.

Underfitting: both losses are high. The model is too small or training is broken.

Overfitting: training loss keeps falling while validation loss rises. The model is memorising.

Today's twist: very large models trained on very large data often keep generalising ("double descent"). More data is the best cure.

Watch the gap between training and validation loss; that gap is overfitting

The stabilisers: regularisation, normalisation and residuals

three small ideas that make deep networks trainable and keep them from memorising

Regularisation still in use

Weight decay: pulls weights toward zero.

Dropout: randomly silences units in training.

Augmentation: crops, colour shifts, noise; "free" extra data.

Early stopping: keep the best validation checkpoint.

Normalisation

Keeps the numbers flowing through the network at a steady scale.

LayerNorm / RMSNorm: the standard in transformers.

BatchNorm: common in older convolutional networks.

Without it, deep models are hard to train.

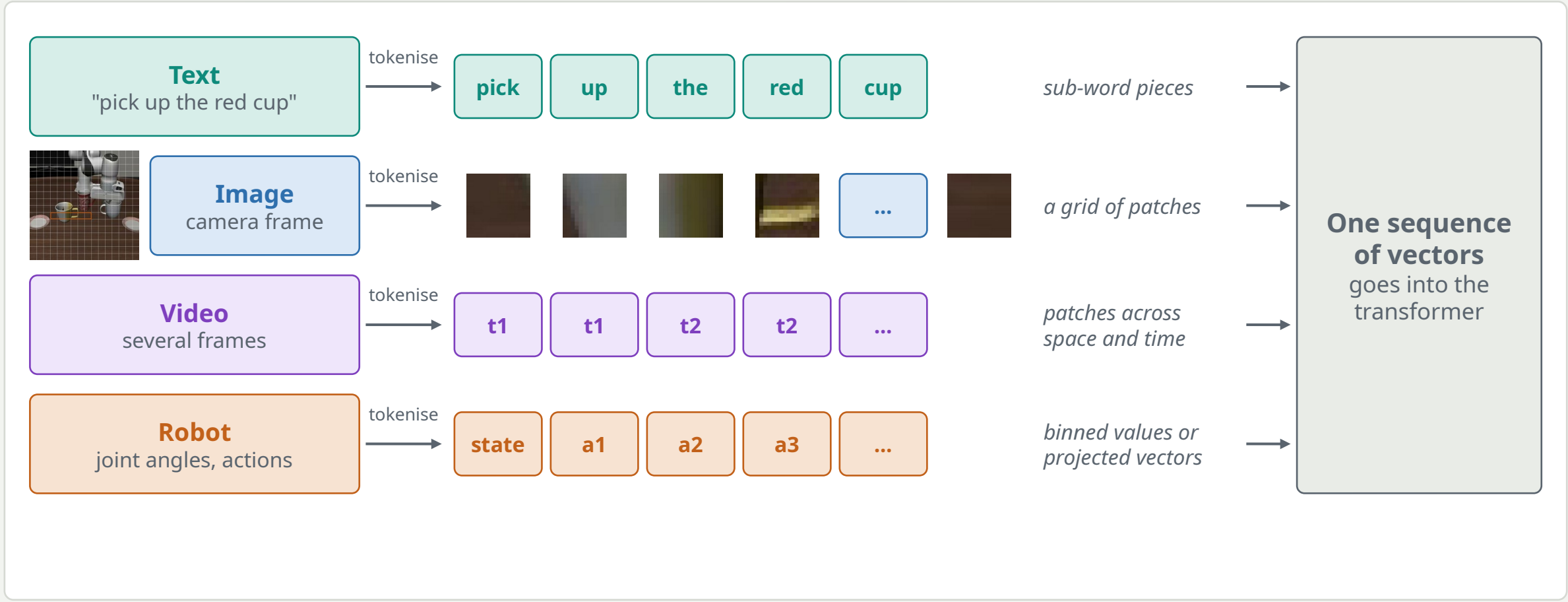
Residual connections

- Each layer adds a correction to its input: output = x + f(x).
- Gradients get a direct path back to early layers.
- This is what allows networks hundreds of layers deep.

Residual connections and normalisation make deep models trainable; regularisation makes them generalise

Everything becomes tokens

a transformer reads one sequence of vectors; the first job is turning each kind of data into that sequence



Once text, images and actions are all tokens, one model can read and write all of them

Attention: every token looks at every other token

attention is how a token gathers the context it needs from the rest of the sequence

How it works

Each token produces a query ("what am I looking for?"), a key ("what do I contain?") and a value ("what do I pass on?").

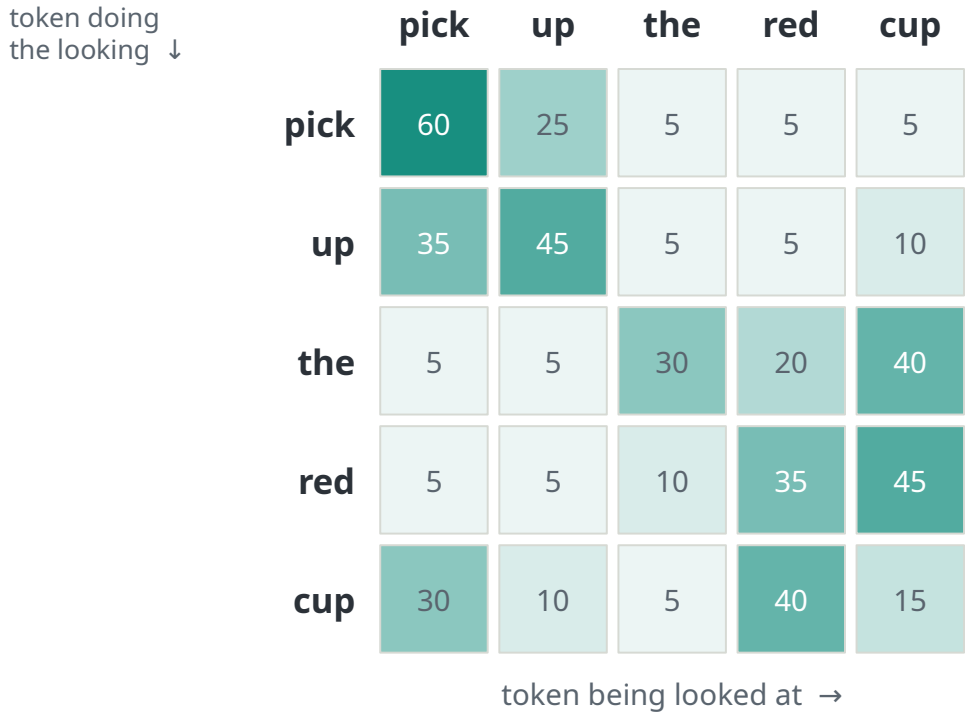
A token's query is compared with every key. High match means high attention weight.

The token's new vector is the weighted sum of the values.

Self-attention: tokens attend within one sequence.

Cross-attention: tokens attend to a second sequence, such as actions attending to image features.

Attention weights for one sentence (illustrative)



Read the last row: "cup" puts most of its attention on "red" and "pick". Its new vector now means "the red cup that is to be picked".

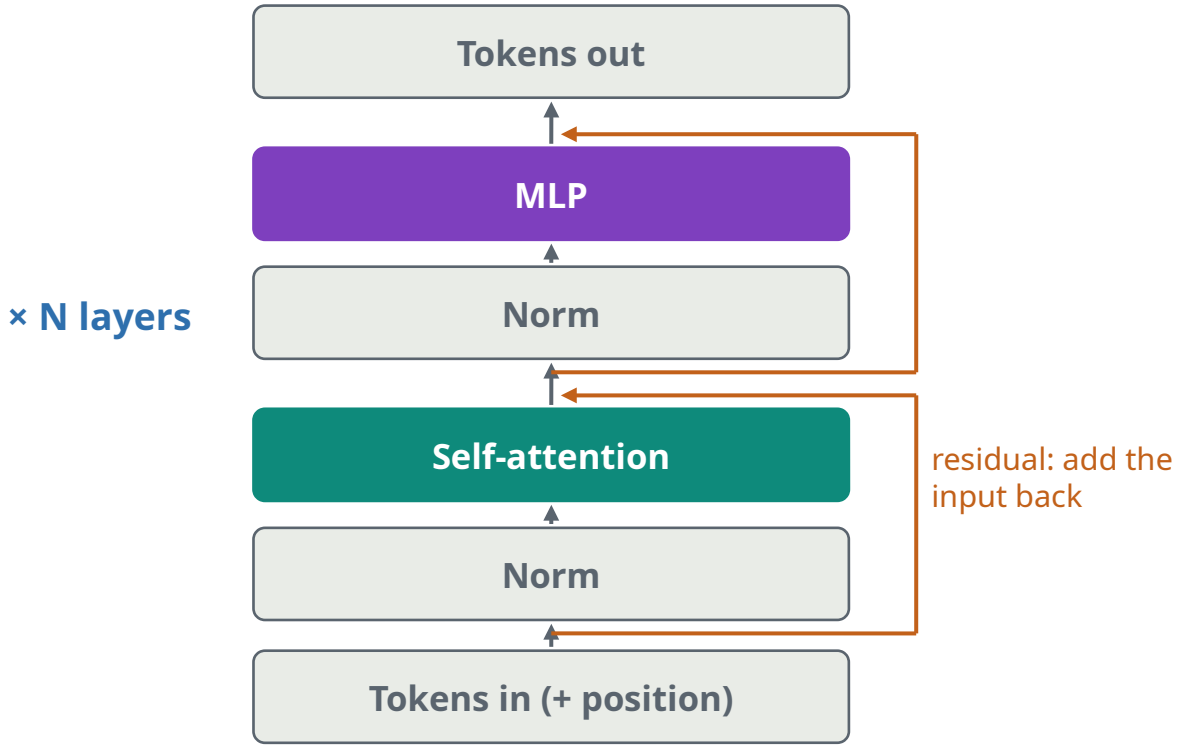
Numbers are percentages and each row sums to 100.

Attention replaces fixed wiring with learned, input-dependent connections

Inside one transformer block

a transformer is the same block stacked many times; understand one and you understand the model

One block, repeated N times



Reading the block

Attention sub-layer: tokens exchange information.

MLP sub-layer: a small network applied to each token separately. Most of the parameters live here.

Norm first: each sub-layer normalises its input ("pre-norm").

Residual add: each sub-layer's output is added back to its input.

Position: attention has no sense of order, so position is injected, today mostly as rotary encoding (RoPE) inside attention.

Attention mixes information between tokens; the MLP processes each token on its own

Three shapes of transformer, and how they are made fast

the block is the same; what changes is what each token is allowed to see

Shape	What a token can see	Good at	Examples of use
Encoder	All tokens, both directions	Understanding an input	Vision encoders, embeddings, classifiers
Decoder	Only earlier tokens (causal)	Generating a sequence step by step	Language models, vision-language models
Encoder-decoder	Decoder attends to the encoder's output	Mapping one sequence to another	Translation, some action policies
KV cache	Stores keys and values of past tokens	Avoids recomputing the prefix	Every generating model at inference
Efficient attention	Same result, less memory and time	Long sequences such as video	Flash attention, sparse or linear variants

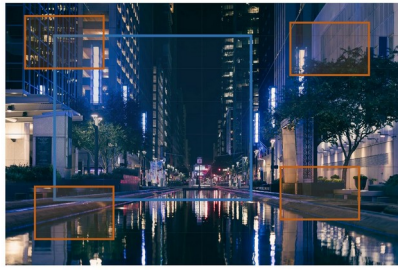
Decoder-only models dominate generation; encoders dominate perception

Self-supervised learning: labels made from the data itself

the model learns useful features by solving a task we can generate automatically, with no human labels

Masked modeling

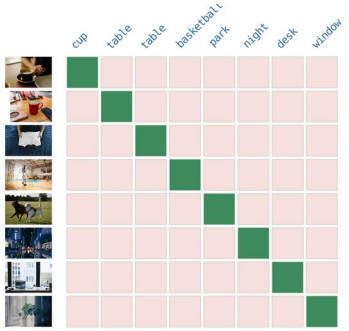
- Hide part of the input; predict the missing part.
- Text: predict masked or next words.
- Images and video: predict masked patches.



I-JEPA: predict the hidden blocks from the visible one

Contrastive learning

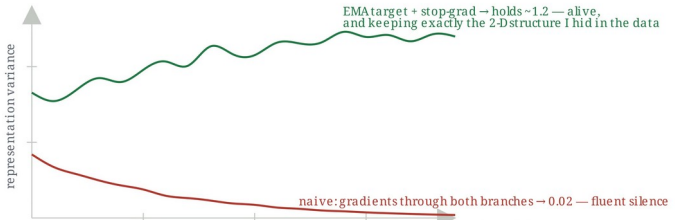
- Take two views of the same thing; make their embeddings match.
- Push embeddings of different things apart.
- Image and caption pairs give CLIP-style models.



each image must pick its own caption out of the batch

Self-distillation

- A student network predicts the output of a slowly updated teacher copy of itself.
- No negatives and no pixel reconstruction.
- Gives strong visual features (the DINO family).



toy run: the slow teacher keeps features from collapsing

Self-supervision is why models can train on billions of images and hours of video

Vision encoders: the eyes of every modern system

a vision encoder turns an image into a set of feature vectors that other models consume

Encoder family	Trained by	Its features are good at	Typical use in robotics
ViT (the architecture)	Any of the methods below	General; a transformer over image patches	The backbone almost everything uses
CLIP / SigLIP	Matching images with captions	Meaning that lines up with language	Vision-language models, open-vocabulary recognition
DINO family	Self-distillation on images only	Fine spatial detail, parts, correspondence	Manipulation features, tracking, depth
Masked autoencoders	Reconstructing hidden patches	Low-level structure	Pretraining on unlabelled video or domain data
Video encoders	Masked or predictive training on clips	Motion and change over time	Dynamics, world models, action recognition

Choose the encoder by what the features must capture: meaning in words, or geometry and detail

Predicting in feature space: JEPA and embeddings

instead of predicting every pixel, predict the representation of what is missing or what comes next

Two ideas on this slide

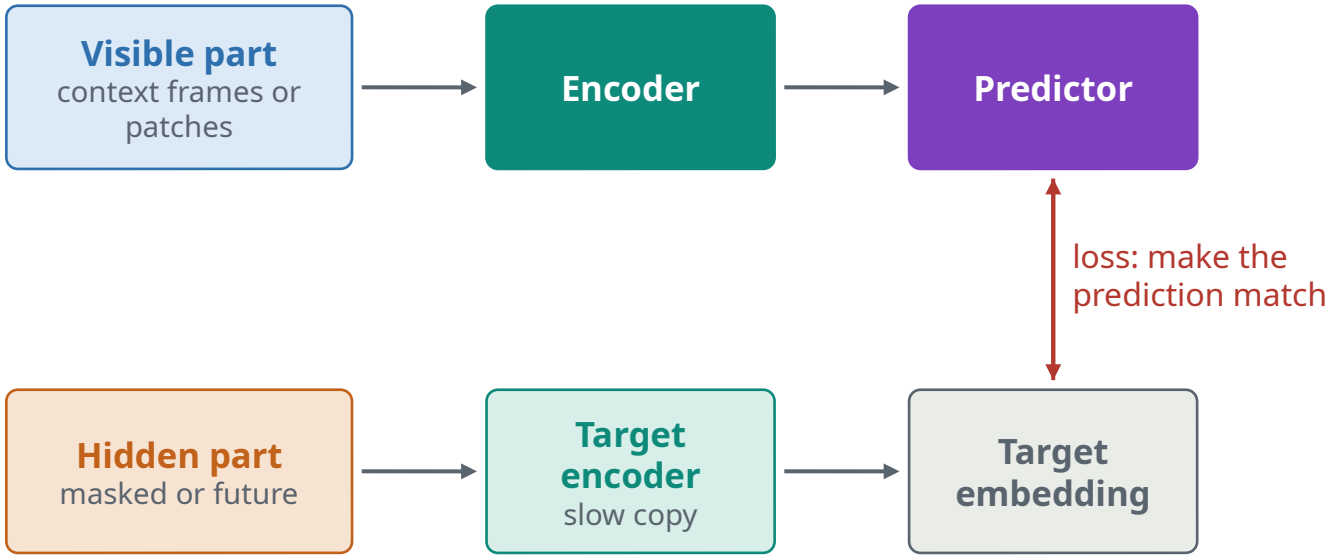
JEPA (joint-embedding predictive architecture): encode the visible part, then predict the embedding of the hidden or future part.

Pixels contain detail that cannot be predicted, such as texture and noise. Predicting embeddings lets the model ignore it.

Embeddings and retrieval: similar things land close together, so search becomes "find the nearest vectors".

Metric learning trains that distance directly; used for re-identification and matching.

JEPA in one picture



No pixels are reconstructed. The loss compares two embeddings.

Good embeddings turn hard problems into nearest-neighbour lookups

Two ways to generate: piece by piece, or all at once

generative models learn a data distribution so they can produce new samples from it

Autoregressive



predict the next token, append it, repeat

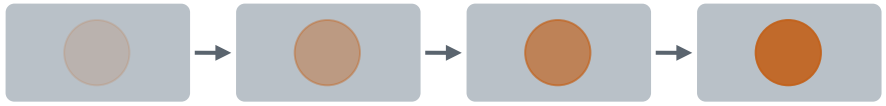
Trained with cross entropy on the next token.

Strengths: simple, exact likelihood, handles variable length.

Limits: one step per token, so long outputs are slow; continuous values must be binned first.

Used for: language, reasoning, tokenised actions.

Diffusion and flow matching



start from noise and refine the whole sample in a few steps

Trained with a regression loss: predict the noise, or the direction toward clean data.

Strengths: sharp continuous outputs; represents several valid answers.

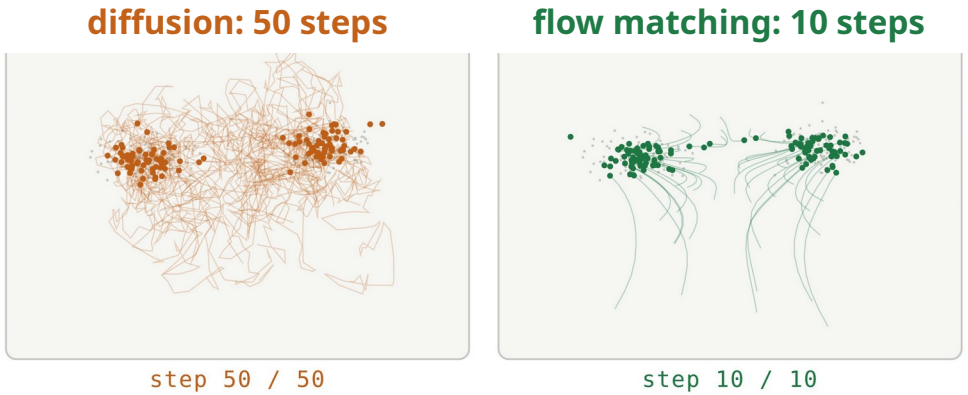
Limits: needs several refinement steps per sample.

Used for: images, video, robot action chunks.

Autoregressive suits discrete sequences; diffusion and flow suit images, video and actions

Diffusion and flow matching, step by step

training destroys data with noise; the model learns to undo it



A real toy model. One small network, trained two ways on 2-D data with two clusters.

Dots are final samples; faint lines are the paths they took from noise.

Diffusion wanders, because noise is re-injected at every step.

Flow matching moves in nearly straight lines, so it needs far fewer steps.

Training

- Mix a real sample with noise at a random level.
- The network predicts the noise or the velocity toward clean data.

Sampling

- Start from random noise.
- Step along the predicted direction, typically 4 to 50 times.

Conditioning

- Feed the network text, images or robot state.
- The same noise then becomes a sample that fits the condition.

The model never sees how to draw a whole sample; it only learns one small denoising step, applied repeatedly

The two training objectives, written out

both are plain regression losses; what differs is the path between noise and data

Diffusion (DDPM)

$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I\right)$$

forward: a fixed noising schedule — no learning

$$\mathcal{L} = \left\| \epsilon - \epsilon_\theta(x_t, t, \mathrm{ctx}) \right\|^2$$

x_0 is a clean sample; x_t is the same sample after t steps of added noise.

$\bar{\alpha}$ (alpha-bar) is the schedule: how much signal is left at step t.

$\varepsilon\theta$ is the network. It sees the noisy sample, the step and the context, and predicts the noise ε that was added.

Sampling runs the chain backwards, removing a little noise at each of many steps.

Flow matching

$$x^\tau = \tau\, x_1 + (1-\tau)\, x_0$$

the path: interpolate noise → data. you CHOOSE it.

$$v = x_1 - x_0$$

its velocity: constant. no schedule to tune.

$$\mathcal{L} = \left\| v_\theta(x^\tau, \mathrm{ctx}, \tau) - (x_1 - x_0) \right\|^2$$

regress the direction — the same safe trick

$$x \leftarrow x + \tfrac{1}{N}\, v_\theta(x, \mathrm{ctx}, \tau)$$

sampling: a deterministic ODE — Euler steps, no dice

x_0 is noise, x_1 is data, and τ runs from 0 to 1 along the straight line between them.

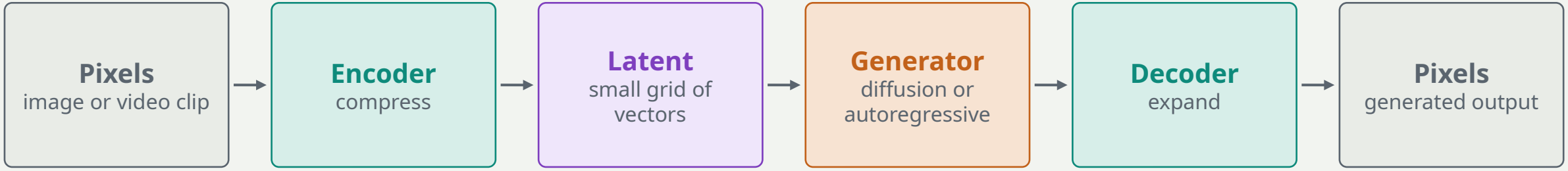
$v\theta$ is the network. It predicts the velocity along that line.

Sampling takes N equal Euler steps from noise to data.

Flow matching keeps diffusion's regression trick and removes the noise schedule

Latent spaces: generate in a compressed world

pixels are expensive, so modern generators work on a compact code and decode it at the end



Why compress

- A 480p video clip is tens of millions of numbers.
- Its latent can be around a hundred times smaller.
- Attention cost falls sharply with fewer tokens.

Two kinds of latent

Continuous (VAE): a grid of real-valued vectors; pairs with diffusion.

Discrete (tokenizer): a grid of codebook indices; pairs with autoregressive models.

What to check

- Reconstruct real data first and look at it.
- Small objects, thin structures and fast motion are lost first.
- Robot grippers and contact points are exactly that kind of detail.

The quality ceiling of a latent generator is set by its autoencoder

Making sampling controllable and fast

three techniques that turn a slow research model into something usable

Guidance

Push samples to follow the condition more strongly.

Classifier-free guidance: run the model with and without the condition and exaggerate the difference.

More guidance: closer to the prompt, less variety.

Few-step sampling

- Each step is a full network pass, so steps cost time.
- Flow matching and better solvers cut 50 steps to under 10.
- Robot action heads commonly use 4 to 10 steps.

Distillation

- Train a student to match many teacher steps in one.
- Gives one-to-four-step generators.
- Trades a little quality for a large speed-up.

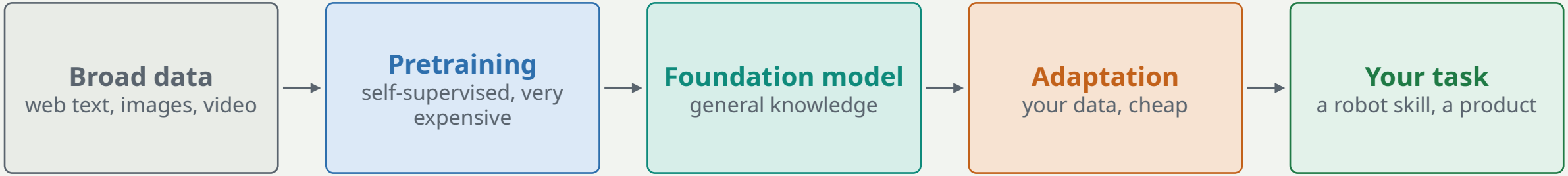
Video generation

- The same recipe with time added to the latent grid.
- Hard parts: consistency across frames, physics, length.
- It is the base technology for world models (section 9).

Few-step sampling is what lets diffusion-style models run inside a robot's control loop

Pretrain once, adapt many times

a foundation model is trained on broad data at great cost, then reused for many tasks at low cost



Language models (LLMs)

- Decoder transformers trained to predict the next token.
- Carry broad knowledge and can follow instructions.

Vision-language models (VLMs)

- A vision encoder feeding a language model.
- Answer questions about images and video.
- The usual starting point for robot policies.

Why it works

- General features transfer to new tasks.
- A little task data goes a long way.
- The risk: the model's gaps and biases transfer too.

Almost no one trains from scratch any more; the skill is choosing a starting model and adapting it well

The adaptation ladder: try the cheap rungs first

ways to get a pretrained model to do your task, ordered from least to most effort

Method	What changes	Data needed	Reach for it when
Prompting	Only the input text	None	The model already knows how; it needs instructions
In-context examples	The input, with a few worked examples	A handful	The output format or style is specific
Retrieval augmentation	The input, with documents fetched at run time	A document store	Answers depend on facts the model lacks
LoRA / adapters	Small added weight matrices; base frozen	Hundreds to thousands	A new skill or domain, on a limited budget
Full fine-tuning	All the weights	Thousands and up	A large shift, such as a new robot or new outputs
Distillation	A smaller student copies the adapted model	The teacher's outputs	The result must run on a small device

Climb only as far as the task needs; each rung costs more data, compute and maintenance

From answering to acting: reasoning, tools and agents

language models are increasingly used to plan and to take steps, not only to reply

Reasoning

- Have the model write intermediate steps before the answer ("chain of thought").
- Trading extra compute at inference for accuracy.
- Robotics uses the same idea: describe the plan, then act.

Tool use

- The model emits a structured call: search, run code, query a database.
- The result is fed back as new input.
- Lets a model act on facts it was never trained on.

Agents

- A loop: observe, decide, act, observe again.
- Useful for multi-step work such as coding or data analysis.
- A robot is an agent whose tools are motors.

Reasoning and tool use are the bridge from chat models to systems that do work, including robots

The control loop: what a robot policy actually does

a policy is a function from what the robot senses to what the robot should do next, called many times a second

Vocabulary

Observation: camera images plus the instruction.

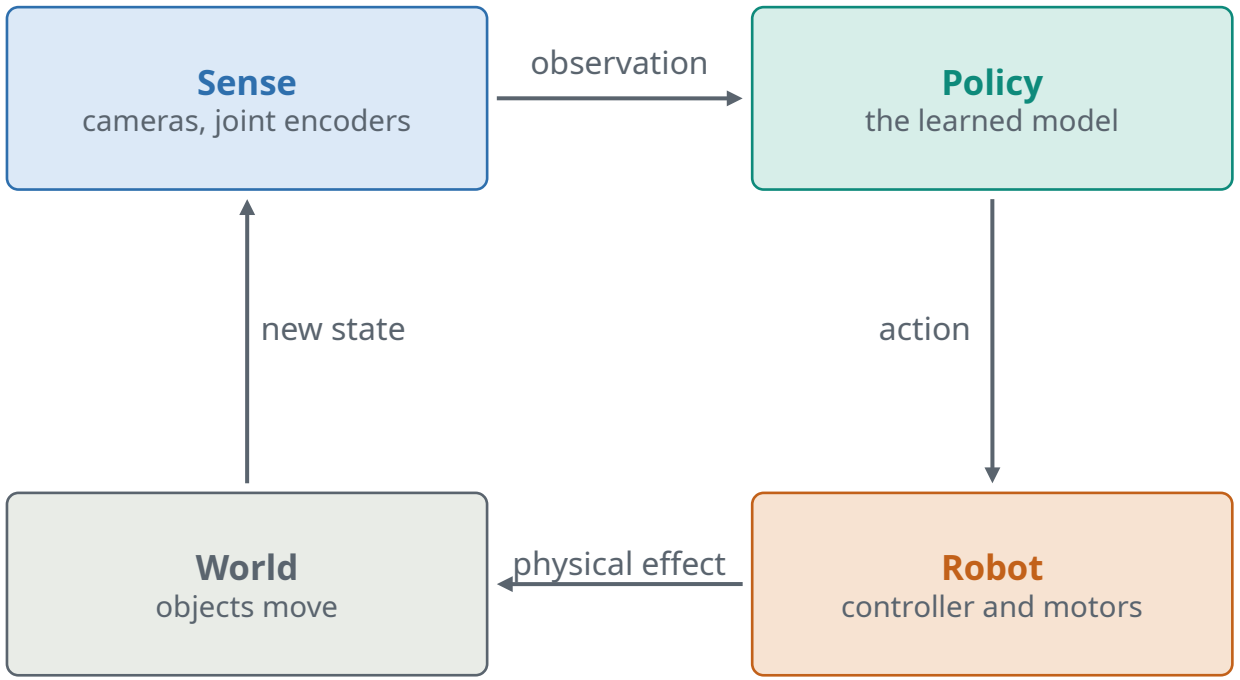
Proprioception: the robot's own state: joint angles, gripper opening.

Action: the command sent to the robot.

Control frequency: how often the loop runs; 10 to 50 Hz for learned policies, 500 Hz and up for the motor controller underneath.

Closed loop: each action changes what the robot sees next.

One cycle of the loop



Unlike a classifier, a policy's outputs change its own future inputs

Action spaces: what exactly does the policy output?

the same motion can be commanded in several ways, and the choice affects learning and transfer

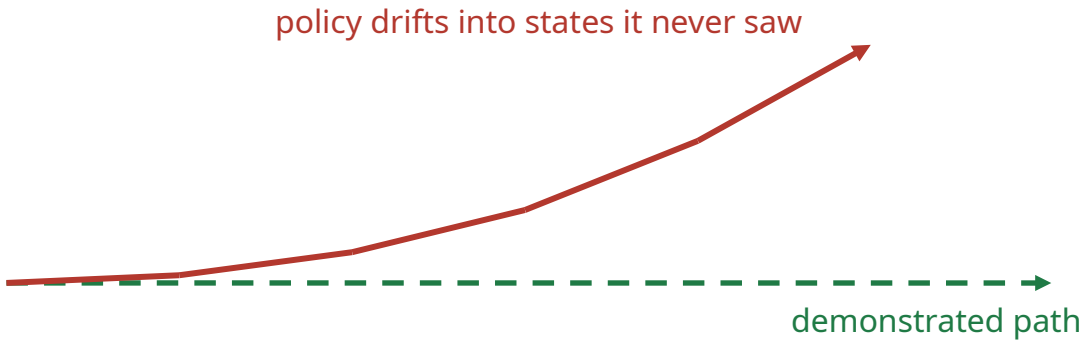
Choice	The policy outputs	Advantages	Watch out for
Joint space	A target for every joint	Exact; no kinematics solver needed	Tied to one robot; hard to transfer
End-effector space	Position and rotation of the hand	Intuitive; transfers across arms	Needs inverse kinematics; rotation formats differ
Absolute	Where to be	Errors do not accumulate	Depends on the workspace and calibration
Delta (relative)	How far to move from here	Generalises across positions	Small errors add up over time
Gripper	Open or close, or a width	Simple	Binary versus continuous conventions vary

Before training on any dataset, find out which action space it uses; mixing them silently breaks a policy

Imitation learning, compounding error and action chunking

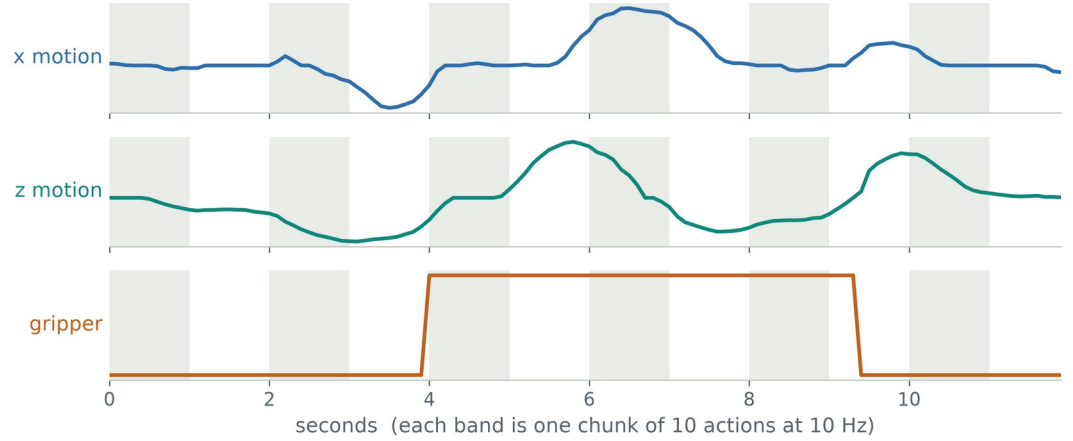
the dominant way to teach robots today: show the task, then train the policy to copy the demonstrations

Compounding error



- Each small error changes the next observation.
- Away from the demonstrations, the policy has no training data.
- Fixes: more varied data, recovery demonstrations, chunking.

Action chunking



- Real actions from one LIBERO episode, cut into chunks.
- One model call produces a whole chunk.
- Re-plan before the chunk ends to stay responsive.

Predicting a chunk of future actions, not one, is the simple trick that made imitation learning work

Reinforcement learning and the classical layer underneath

learned policies do not replace control engineering; they sit on top of it

Where RL works

- Legged locomotion trained in simulation, then transferred.
- Tasks with a clear reward and a good simulator.
- Fine-tuning an imitation policy to raise its success rate.
- Games and other fully simulated worlds.

Where RL struggles

- Needs millions of trials; real robots are slow and break.
- Rewards are hard to specify for everyday tasks.
- Simulated contact, cloth and liquids differ from reality.
- Unsafe exploration around people and equipment.

The stack on a real robot

Learned policy
10 to 50 Hz: what to do

Kinematics and planning
turn targets into joint motion

Joint controller
500 Hz and up: track the target

Motors and safety limits
torque, speed, emergency stop

Imitation leads in manipulation, RL in locomotion, and classical control runs under both

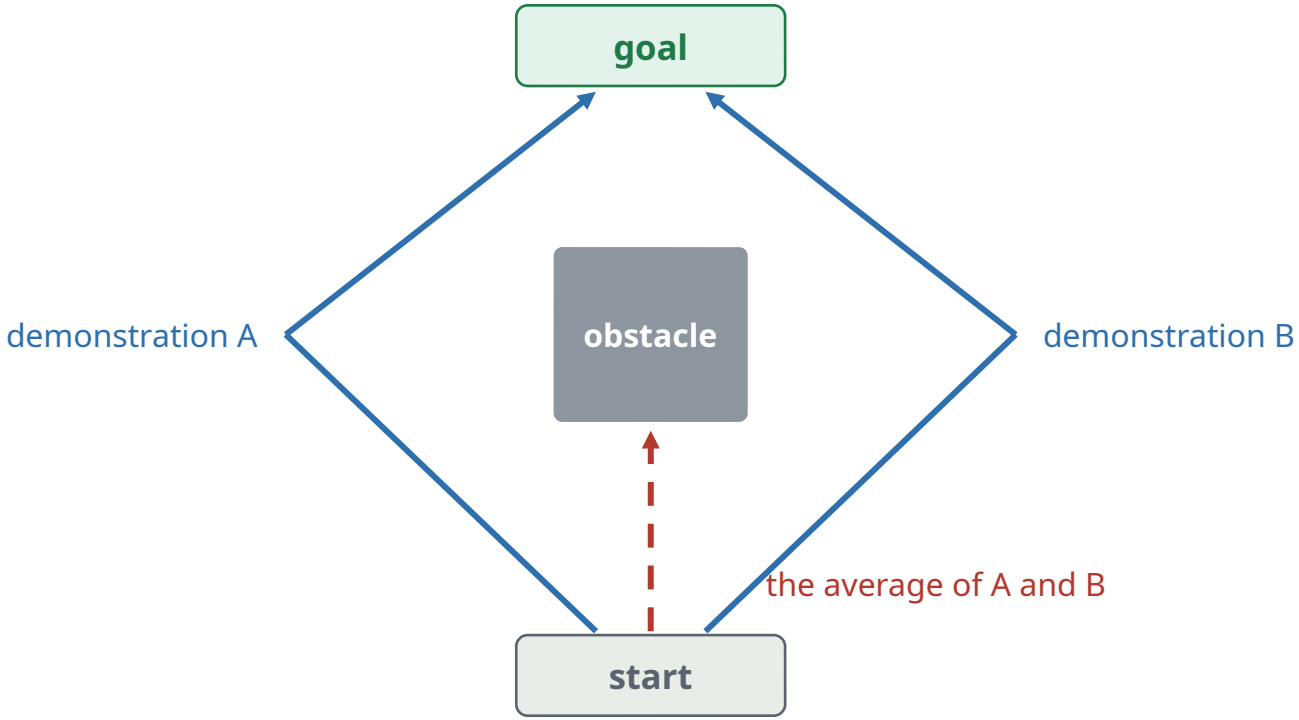
Diffusion and flow policies: generating actions like images

the generative recipe from section 5, applied to a chunk of future actions conditioned on what the robot sees

Why not plain regression?

- Demonstrations are multi-modal: one person goes left around an obstacle, another goes right.
- Regression with MSE predicts the average: straight into the obstacle.
- A diffusion or flow policy samples one consistent option.
- Input: recent images and robot state. Output: a chunk of actions, denoised from noise in a few steps.

Two valid demonstrations, one bad average



Generative action heads solved the "average of two good answers is a bad answer" problem

Anatomy of a vision-language-action model (VLA)

a VLA is a vision-language model that has been taught to output robot actions



scene + wrist

Vision encoder
SigLIP, DINO

Instruction
"put the cup on the shelf"

Text tokenizer

Robot state
joint angles, gripper

State projector

VLM backbone
a pretrained language model reads all the tokens together

Action head
tokens, or a flow / diffusion network

action chunk

Pretrained on web data: the vision encoder and the VLM backbone.

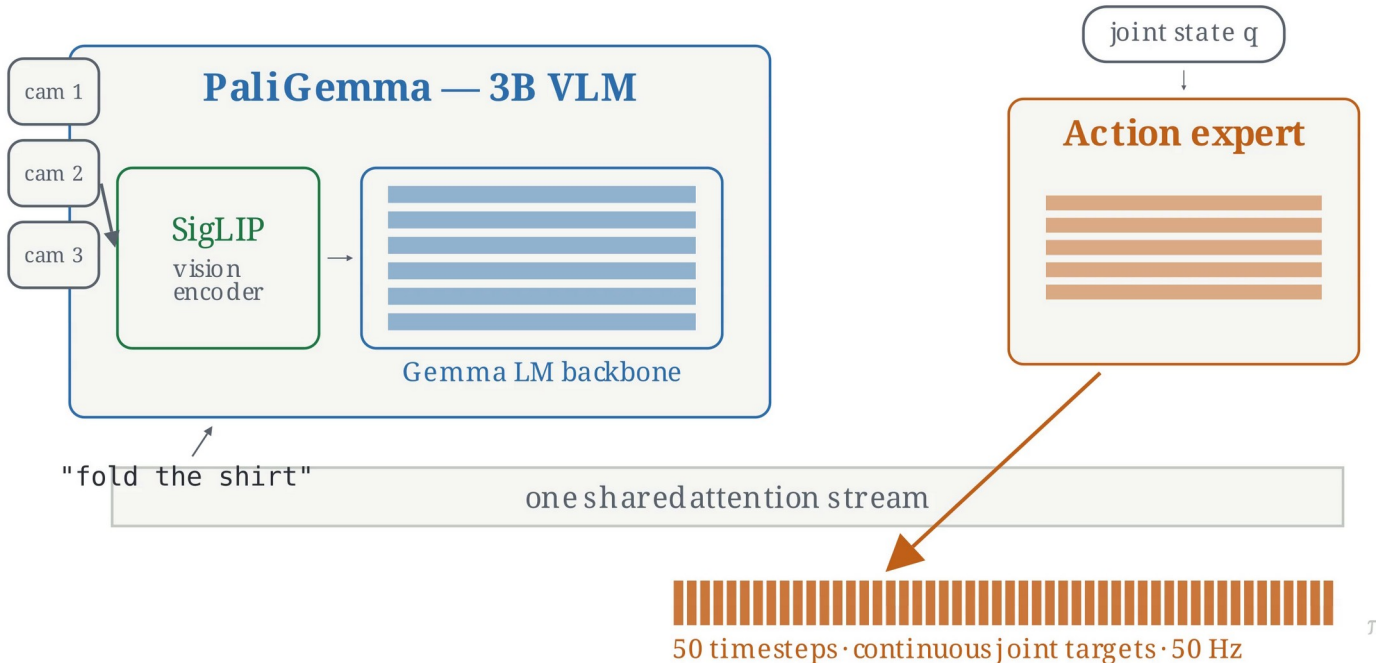
Trained on robot data: the state projector, the action head, and usually a fine-tune of the backbone.

A VLA inherits what the internet taught the VLM, and learns only the acting part from robot data

A real example: π_0

one published VLA, with its parts drawn out

Two transformers sharing one attention stream



Reading the figure

Backbone: PaliGemma, a 3-billion-parameter VLM: the SigLIP vision encoder plus the Gemma language model.

Action expert: a separate transformer of about 300 million parameters, trained with flow matching.

Inputs: up to three camera views, the instruction and the joint state.

Output: a chunk of 50 continuous actions: one second of motion at 50 Hz.

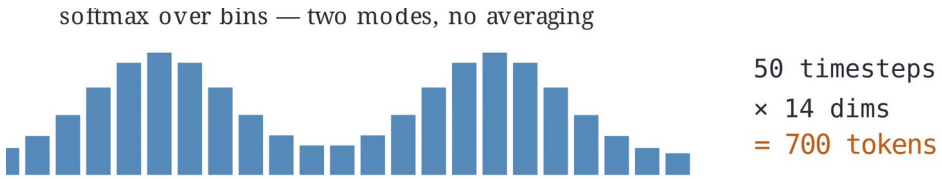
Coupling: the expert attends to everything the VLM computed.

Most of the parameters understand the scene; a small expert turns that understanding into motion

Two ways to output actions: tokens or a continuous head

the main design choice inside a VLA, with real consequences for speed and precision

Discrete action tokens



How: bin each action value; predict bins as tokens with cross entropy.

Good: no new architecture; works with any language model; trains stably.

Costs: one token per value is slow; binning limits precision.

Improving: learned tokenisers that compress a whole chunk.

Continuous action head



How: a small network denoises the whole action chunk, conditioned on backbone features.

Good: smooth, precise, a few steps for many actions.

Costs: an extra module and loss; care needed so it does not degrade the backbone.

Used by: most recent high-frequency manipulation policies.

Token outputs keep the model simple; continuous heads give speed and precision. Many current systems use both

Three trends shaping current policies

how the field is stretching VLAs toward harder, longer and more varied tasks

Dual-system designs

- A large, slow model understands the scene and the instruction a few times a second.
- A small, fast model turns that into actions at control rate.
- Often compared to "thinking slow, acting fast".

Embodied reasoning

- Before acting, the model states the sub-task, the objects and where they are.
- Helps on long, multi-step tasks and makes failures explainable.
- Costs extra tokens, so it is often run less often than actions.

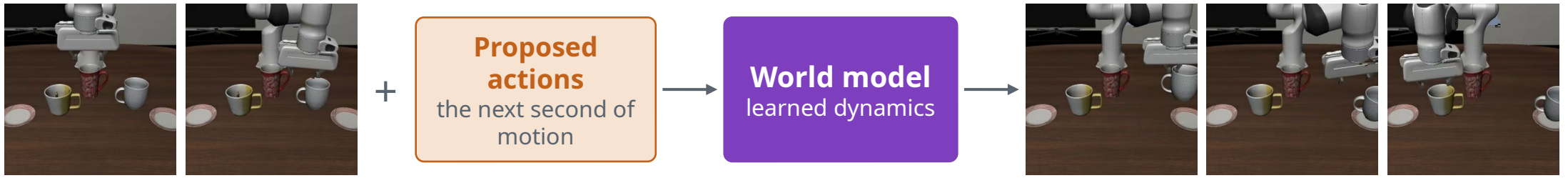
Cross-embodiment training

- One model trained on data from many different robots.
- Shared skills transfer; a new robot needs less data.
- Requires handling different cameras, action spaces and rates.

The direction of travel: one model, many robots, with slow thinking and fast acting separated

World models: learning to predict what happens next

a world model takes the current situation and a proposed action, and predicts the outcome



current observation (real camera frames)

predicted future (illustrated with later frames of the same episode)

Why robots want one

- Try actions in imagination before the real world.
- Video is plentiful; it teaches physics without action labels.

What it must get right

- Contact, occlusion and object permanence.
- Consequences of the action, not just plausible video.

How it is trained

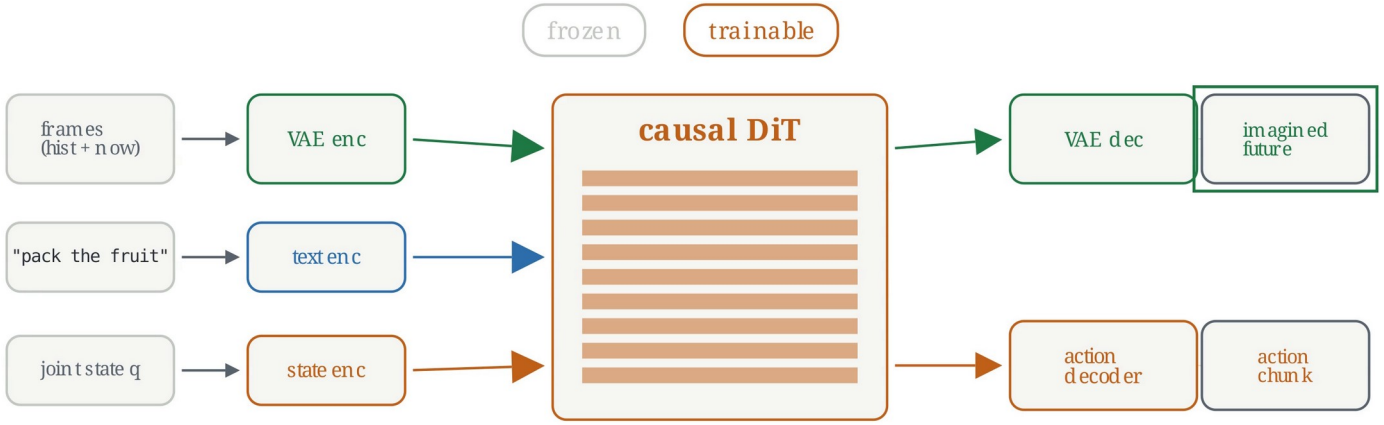
- Video generation objectives, conditioned on actions.
- Pretrained on general video, then tuned on robot data.

A policy answers "what should I do?". A world model answers "what will happen if I do this?"

World-action models: predict the future and act, in one model

instead of a separate policy and world model, train one network to produce both the future and the actions that cause it

Example wiring: DreamZero (NVIDIA GEAR)



One causal diffusion transformer (DiT) reads past frames, the instruction and the joint state. Its output is decoded twice: into imagined future frames and into an action chunk.

Design choices

Joint training: one backbone, two outputs: future observations and actions.

Why it may help: predicting the future is a dense training signal that plain imitation lacks.

Pixel prediction: interpretable and visual, but spends capacity on appearance.

Latent prediction: cheaper and focused on what matters, but harder to inspect.

At run time: the future head can be skipped to save compute.

The bet: a model that can predict what its actions will cause has learned something a pure policy has not

Three uses for a world model, and one caution

what teams actually do with learned simulators today

Evaluate policies

- Roll a policy out inside the model, not on a robot.
- Cheap, fast and safe.
- Useful for ranking checkpoints before real tests.

Generate data

- Produce new scenes, lighting and viewpoints.
- Augment scarce robot demonstrations.
- Generated data needs actions that match the video.

Plan

- Imagine several action sequences.
- Score the outcomes; execute the best.
- Slow today; used for high-level decisions.

The caution

- Models hallucinate plausible but wrong physics.
- Errors grow with rollout length.
- Validate: does success in the model predict success on the robot?

A world model is only as useful as it is faithful; always check it against real outcomes

The data pyramid: plentiful at the bottom, relevant at the top

robotics has no internet-scale dataset of robot actions, so models are built from layers of different data

Reading the pyramid

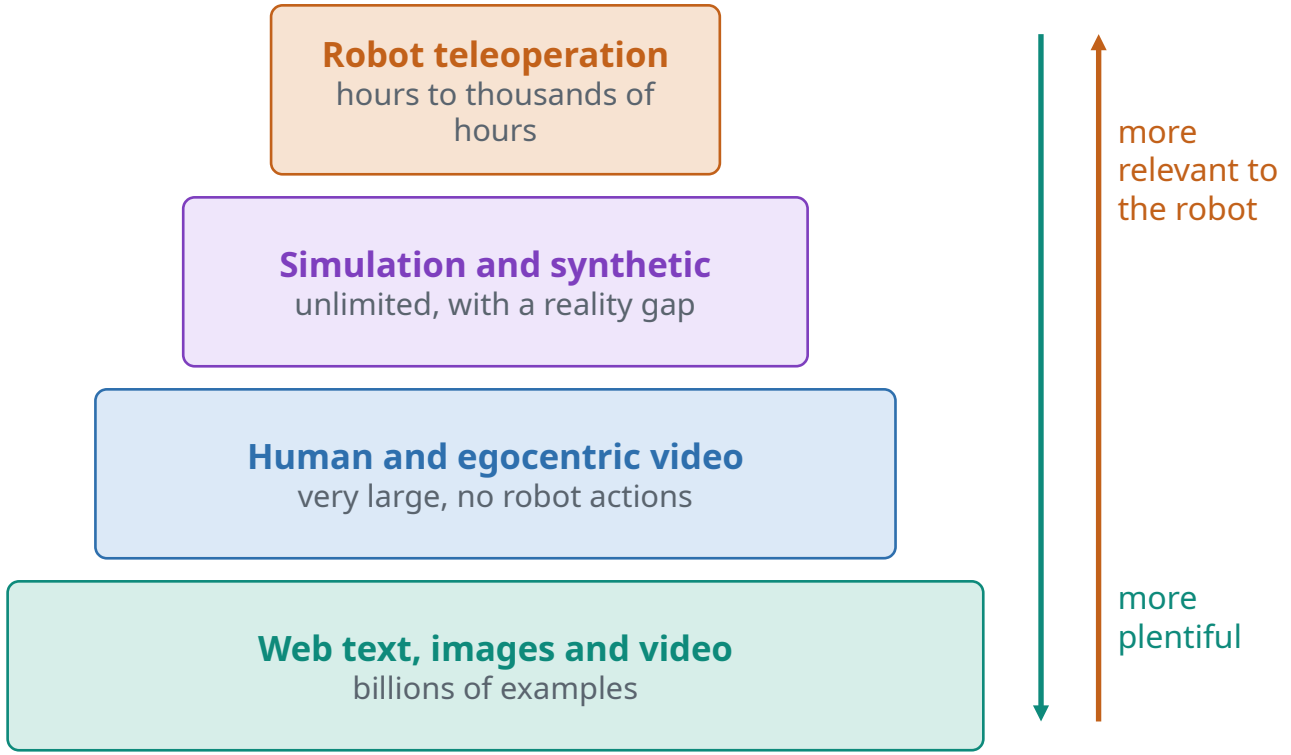
Web data teaches language, objects and common sense. No actions.

Human video shows how tasks are done by hands. Actions must be inferred.

Simulation and synthetic data have perfect labels and unlimited scale, but differ from reality.

Teleoperation is exactly the right data: the robot's own sensors and actions. It is slow and costly to collect.

Sources of training data



Use the cheap layers to learn about the world, and the expensive layer to learn how to act

Collecting demonstrations and the datasets you will meet

how robot data is gathered, and the public collections and formats that have become standard

Teleoperation interfaces

Leader-follower arms: move a small copy of the arm; the robot mirrors it.

VR headsets and controllers: track the operator's hands.

Handheld grippers: collect data with no robot at all.

Exoskeletons and gloves: for humanoids and dexterous hands.

Public datasets

Open X-Embodiment: pooled data from many labs and robots.

DROID: one arm setup, many buildings and scenes.

LIBERO and similar: simulated benchmark suites.

Many more are released each year.

Formats and tooling

LeRobot format: episodes as video plus tables; widely used.

RLDS: the format behind Open X-Embodiment.

Each records observations, actions, timestamps and task text.

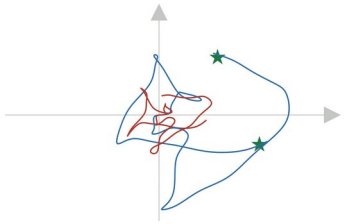
Conversion between them is a routine chore.

Know your dataset's cameras, action space and control rate before you train on it

Simulation, the reality gap and data quality

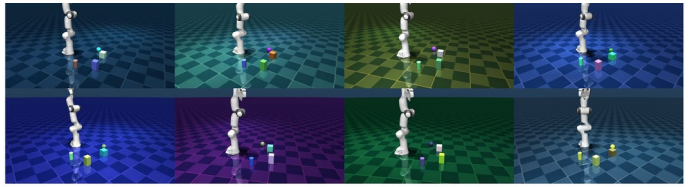
simulated data is cheap and perfectly labelled, but a policy trained only in simulation often fails on the real robot

A wrong simulator, measured



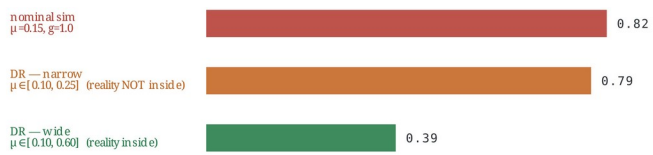
blue: what the simulator predicted; red: what really happened

Domain randomisation



train across many random lightings and textures

Does randomising help?



mean miss: plain sim 0.82, narrow range 0.79, wide range 0.39 (lower is better)

The reality gap

- Rendered images look different from camera images.
- Simulated contact and friction are approximations.
- Real sensors have delay and noise.

Synthetic and generated data

- Generative models re-light, re-texture and re-stage real demonstrations.
- Cheaper than re-recording, if the actions stay valid.

Data quality checklist

- Are demonstrations smooth and successful?
- Is the task description correct for each episode?
- Are cameras and timestamps aligned?

More data only helps if it is good data: consistent, varied and correctly labelled

Classical metrics in one slide: the confusion matrix

for any yes-or-no decision, four counts tell the whole story; the named metrics are ratios of them

Predictions against reality

	model says YES	model says NO
really YES	**True positive** TP: correct detection	**False negative** FN: a miss
really NO	**False positive** FP: a false alarm	**True negative** TN: correct rejection

The ratios

Precision = TP / (TP + FP). When the model says yes, how often is it right?

Recall = TP / (TP + FN). Of the real positives, how many did it find? Also called sensitivity.

Specificity = TN / (TN + FP). Of the real negatives, how many did it reject?

F1: the harmonic mean of precision and recall.

ROC / AUC: the trade-off across all decision thresholds.

Calibration: does "90% confident" mean right 90% of the time?

Accuracy hides which kind of mistake the model makes; precision and recall show it

Evaluating on a real robot: count your trials

robot evaluations are slow, so sample sizes are small, and small samples have wide error bars

Rules for a fair test

Write the protocol first: tasks, object positions, what counts as success.

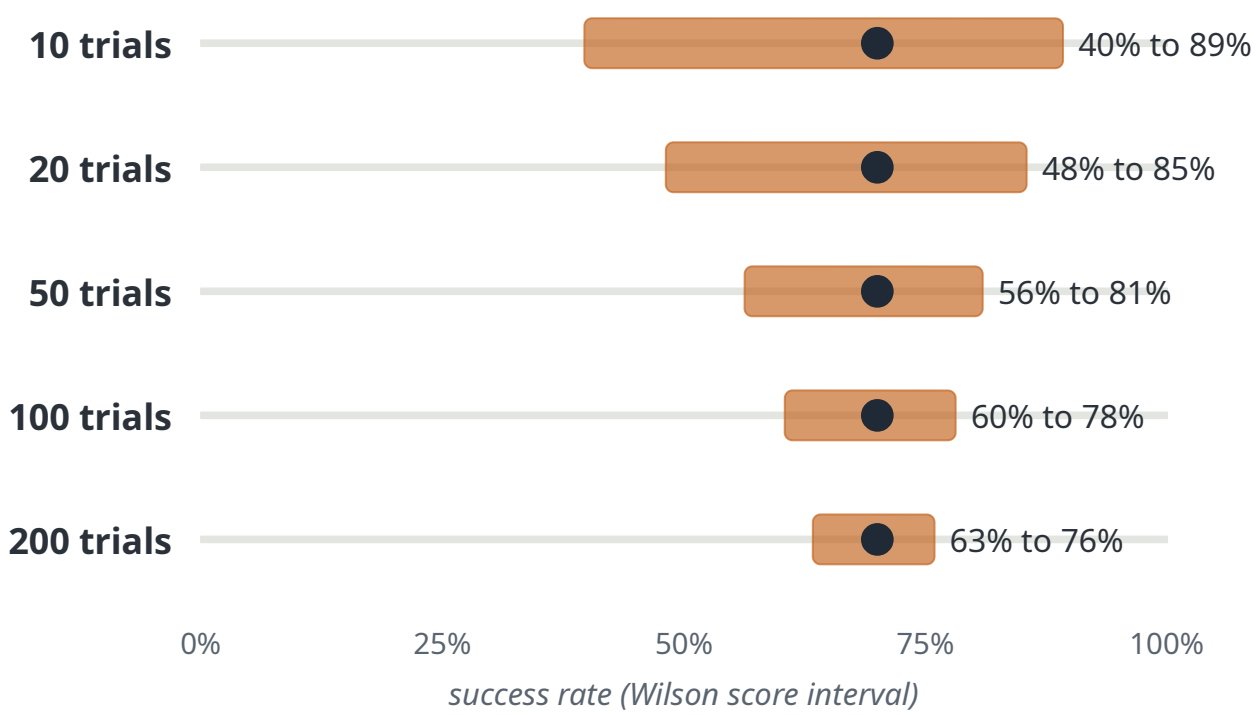
Same conditions for every model: same positions, lighting, time limit.

Enough trials: the chart shows how slowly intervals shrink.

Blind where possible: the operator should not know which model is running.

Report partial progress: which stage failed, not only pass or fail.

95% interval when the measured success rate is 70%



Ten trials cannot tell 70% from 50%. Report the number of trials and an interval, not just a percentage

What a single success rate hides

an average over tasks can conceal exactly the failures that matter

Generalisation axes

- New object positions.
- New objects of a known kind.
- New scenes, lighting and camera angles.
- New instructions and wording.
- Report each axis separately.

Benchmark traps

Saturation: everyone scores 95% +; differences are noise.

Contamination: test tasks were in the pretraining data.

Overfitting to the benchmark: tuned on the test tasks.

Sim only: simulation scores may not transfer.

Failure analysis

- Watch the videos of failures.
- Label each: perception, grasp, wrong object, timing, recovery.
- Count the labels; fix the largest group.
- Stratify by task, object and scene.

Break results down before you believe them, and look at the failures yourself

A recipe for training neural networks

Andrej Karpathy's 2019 recipe is still the best checklist for starting any training project

1 Know the data

- Look at hundreds of samples.
- Find duplicates, bad labels, imbalance.

2 Skeleton + baseline

- Full train and eval loop with a tiny model.
- Fix the random seed.
- Check the loss at the start is what you expect.

3 Overfit

- Make the model big enough to memorise the training set.
- If it cannot, there is a bug.

4 Regularise

- Add data first.
- Then augmentation, dropout, weight decay.
- Trade training loss for validation loss.

5 Tune

- Search learning rate and a few key settings.
- Change one thing at a time.
- Train to convergence.

Build up from something simple that works; never start with the full system

The engineering around a training run

large runs are software projects; these practices keep them fast, comparable and recoverable

Mixed precision

- Compute in 16-bit, keep sensitive parts in 32-bit.
- Roughly halves memory and speeds training.
- Watch for overflow and NaNs.

Distributed training

- Split each batch across several GPUs.
- Larger effective batch, shorter wall-clock time.
- Data loading often becomes the bottleneck.

Experiment tracking

- Log every run: config, code version, metrics, samples.
- Tools: Weights & Biases, MLflow, TensorBoard.
- Name runs so you can find them in a year.

Pin and diff configs

- Set every hyperparameter explicitly.
- Defaults drift between code versions.
- Diff the config against the reference run before launch.

Checkpointing

- Save model, optimizer and step regularly.
- Resume must give the same result as an uninterrupted run.
- Keep the best checkpoint by validation metric.

Convergence

- A run is done when the validation curve has flattened.
- Not when the step budget ends.
- Check the learning rate was still alive at the plateau.

A result you cannot reproduce or compare is not a result

Making the model fit: quantisation, pruning, distillation

a model that needs a data-centre GPU is no use on a battery-powered robot

Three ways to shrink a model

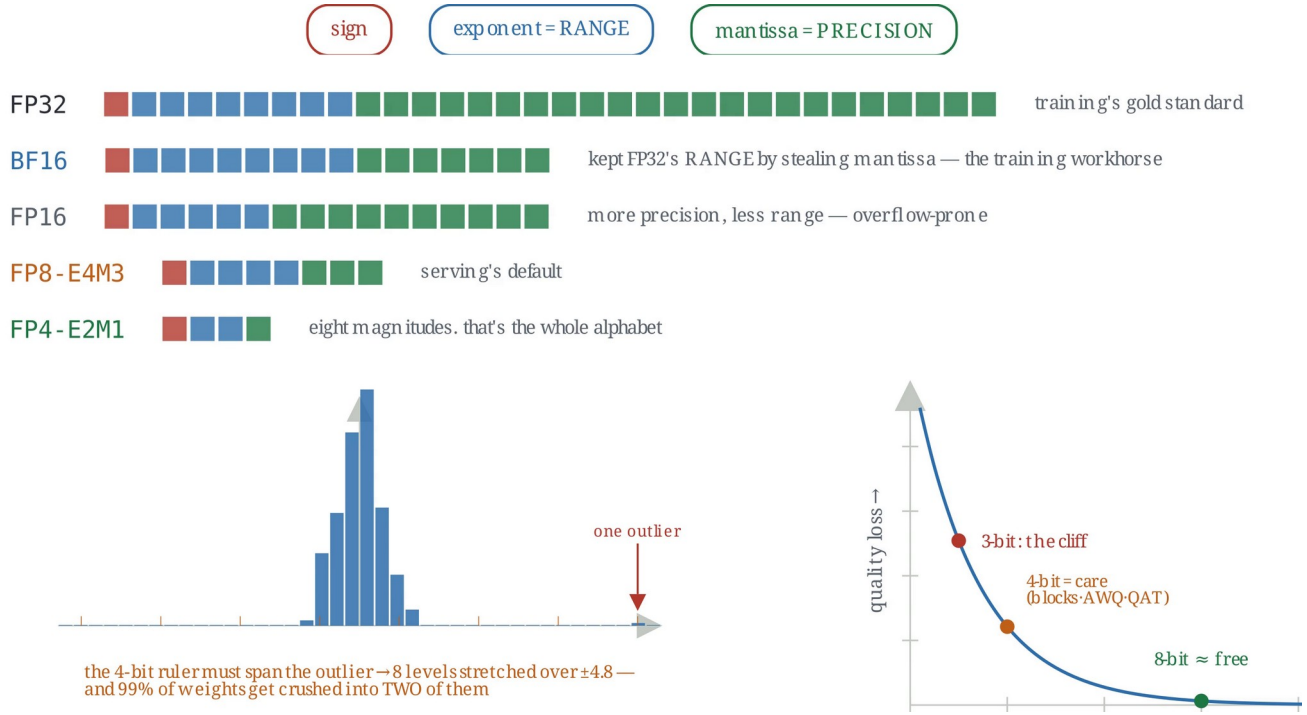
Quantisation: store and compute with fewer bits per number. FP16 is usually free; INT8 needs calibration.

Pruning: remove weights, heads or layers that contribute little.

Distillation: train a small student to copy a large teacher's outputs.

Check: small numerical errors can compound over a control loop. Test closed-loop success, not just output difference.

Number formats, bit by bit



one outlier stretches the scale and crushes the rest

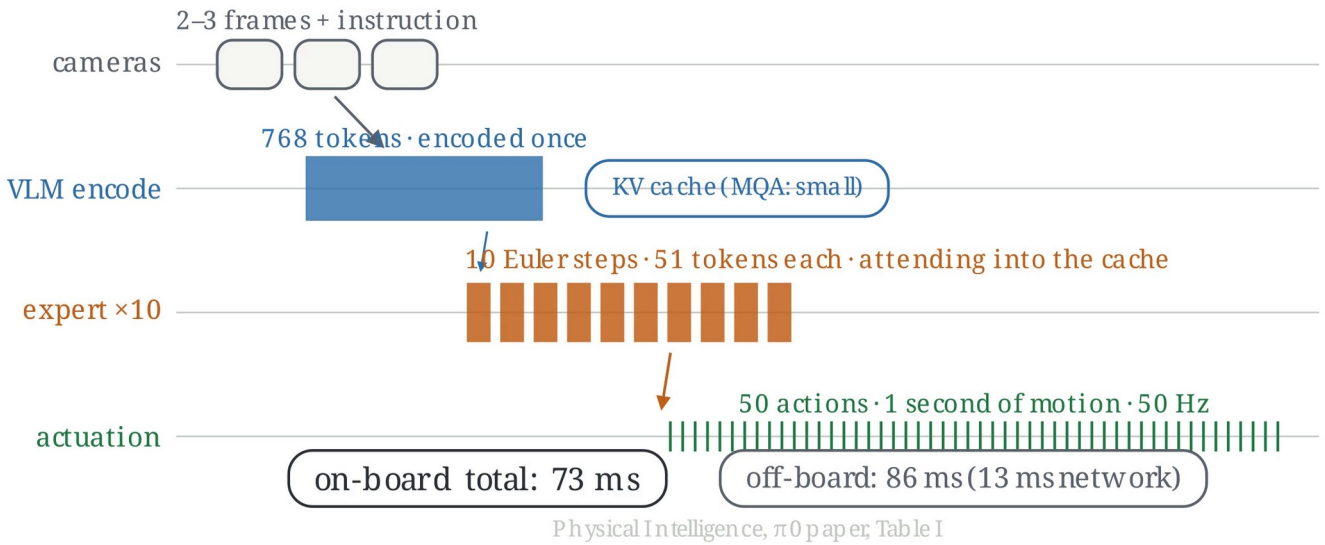
loss versus bits (schematic)

Quantisation is the first tool to reach for; always re-measure task success afterwards

The latency budget: answer before the world moves on

the model's answer always arrives late; the design goal is to make that harmless

One control cycle of π_0, timed (from the paper's timing table)



73 ms of computation yields 50 actions: one second of motion at 50 Hz. The next chunk is ready long before this one runs out.

Four ways to cope

Chunk and overlap. Each call returns a second of motion, so the next call can run while this one executes.

Asynchronous inference. Motion never pauses to wait for the model.

Real-time chunking. Plan the new chunk knowing it will arrive late, and blend it with actions already committed.

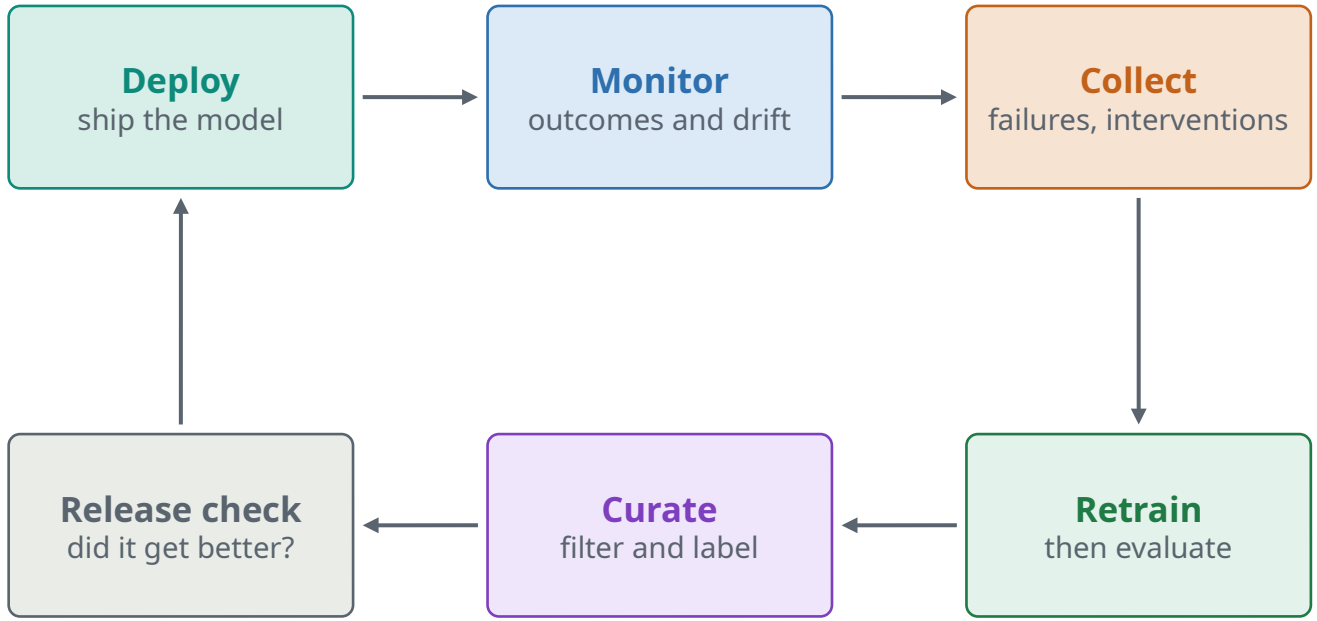
Split the work. Encode the images once and cache the result; only the small action expert runs repeatedly.

Latency is a correctness issue: an action computed from an old image is an action for a world that no longer exists

The data flywheel: the best data comes from deployment

a deployed robot meets situations the training set never covered; capture them and feed them back

The loop



What to monitor

Task outcomes: success, failure, time taken, human interventions.

Input drift: new lighting, objects, camera position.

Model signals: low confidence, unusual actions, large corrections.

System health: latency, dropped frames, temperatures.

Failures and interventions are the most valuable episodes to keep.

The team that turns field failures into training data fastest improves fastest

Looking inside the model, and knowing when not to trust it

tools for asking what a model has learned and whether the current input is one it can handle

Probing

- Train a small classifier on internal features.
- If it can read out object position, the model encodes it.
- Cheap first check of what a layer knows.

Attention and saliency

- Show which pixels or tokens influenced an output.
- Good for spotting shortcut learning.
- Suggestive, not proof.

Sparse autoencoders

- Decompose activations into many simple features.
- Each feature often has a readable meaning.
- An active research area.

Ablation

- Remove or zero a component and measure the change.
- The most direct causal evidence.
- Also: mask an input, such as a camera.

Uncertainty

- Estimate how sure the model is.
- Ensembles, sampling several outputs, calibrated scores.
- Lets the system ask for help.

Out-of-distribution detection

- Flag inputs unlike the training data.
- Compare embeddings with those seen in training.
- Trigger a safe stop or a human check.

Interpretability tools suggest hypotheses; an ablation or a controlled test confirms them

Physical safety: layers outside the learned model

a learned policy will sometimes be wrong, so safety has to be enforced outside it

Principles

Defence in depth: several independent layers, each able to stop the robot.

Lower layers are simpler: limits in firmware and hardware are easier to verify than a neural network.

Fail safe: on lost communication or stale commands, stop.

Limit energy: cap speed and force near people; collaborative robots are built for this.

Test the stop: exercise the emergency stop regularly.

Safety layers, outermost first

Human supervisor
watches, can take over or stop

Runtime monitor
workspace bounds, anomaly and out-of-distribution checks

Learned policy
proposes actions; assumed fallible

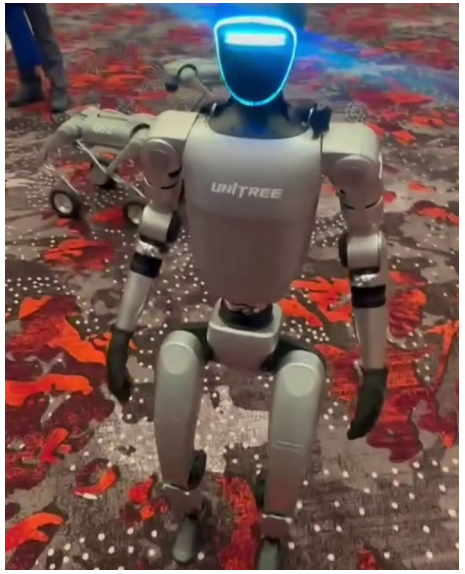
Low-level controller
joint, velocity and force limits

Hardware
emergency stop, torque limits, brakes

Never rely on the policy to be safe; put hard limits underneath it and a human above it

Four robots you will meet

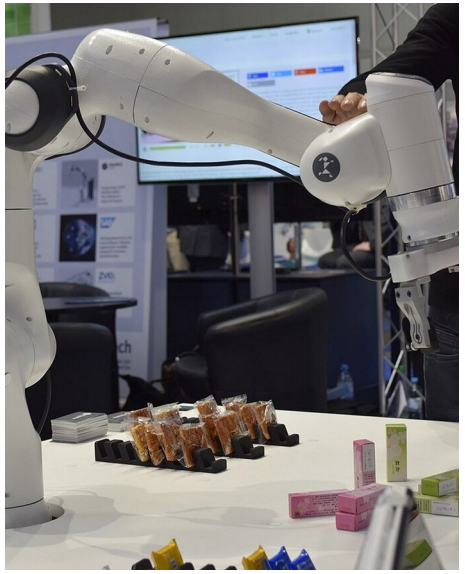
the methods in this deck run on machines like these



Unitree G1

Humanoid: whole-body control, two arms and hands.

Photo: Sayanesy, CC0



Franka research arm

Seven joints with torque sensing. The standard lab arm, and the arm in LIBERO.

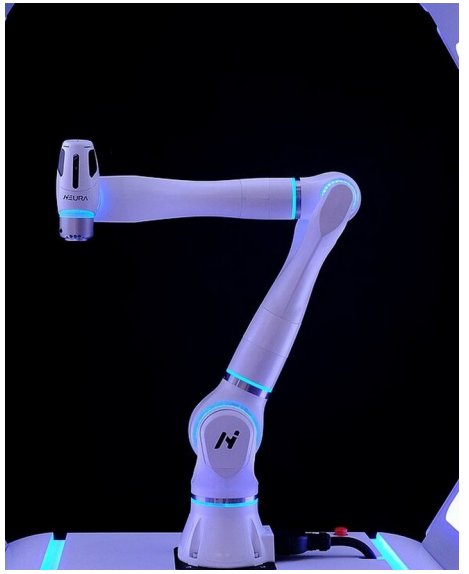
Photo: Ims, CC BY-SA 4.0



Flexiv Rizon

Seven-joint adaptive arm built around force control.

Photo: MichelleACipriano, CC BY-SA 4.0



NEURA MAiRA

Collaborative arm with built-in sensing, for work beside people.

Photo: Anna Schneider2021, CC BY-SA 4.0

Different bodies, same recipe: cameras in, action chunks out

The bodies: what robot are we actually controlling?

the same learning methods run on very different machines; each body sets different constraints

Embodiment	Typical degrees of freedom	Strengths	Challenges for learning
Arms and collaborative robots	6 or 7, plus gripper	Precise, mature, safe to work beside	Fixed base limits reach and the tasks available
Parallel grippers	1	Simple, robust, easy to learn	Cannot re-orient objects in the hand
Dexterous hands	12 to 20+	Human-like manipulation	Hard to teleoperate; fragile; large action space
Mobile manipulators	Arm plus a moving base	Large workspace	Base and arm must be coordinated
Quadrupeds	12	Rough terrain, stable	Little manipulation unless an arm is added
Humanoids	20 to 50+	Fit human spaces and tools; can learn from human video	Balance, whole-body control, cost, data collection

The body decides the action space, the data you can collect and what can go wrong

Sensing: what the robot knows about the world and itself

a policy can only use what its sensors provide; choosing and placing sensors is part of the design

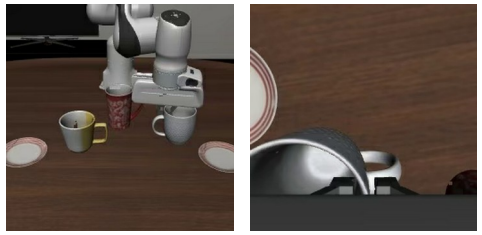
RGB cameras

The main input for learned policies.

Scene cameras: the wider context.

Wrist cameras: close-up of the grasp; large gains in precision.

Placement must match the training data.



scene camera

wrist camera

Depth

- Stereo, structured light or lidar.
- Gives geometry directly.
- Noisy on shiny and transparent objects.
- Many policies skip it and learn depth from RGB.

Proprioception

- Joint positions, velocities and torques.
- Always available and precise.
- Policies can over-rely on it and ignore the cameras.

Touch and force

Force-torque sensors: contact forces at the wrist.

Tactile sensors: pressure and slip at the fingertips.

Essential for insertion, fragile objects and anything occluded by the hand.

If a human teleoperator could not do the task with the robot's sensors, the policy cannot either

What to remember

If the detail fades, keep these six

Data sets the ceiling

Scale, variety and quality matter more than the architecture.

Tokens, attention, residuals

One block, stacked, now reads text, images, video and actions.

Pretrain, then adapt

Start from a foundation model and climb the adaptation ladder only as far as needed.

Actions come in chunks

VLAs with flow or diffusion heads are the current standard for manipulation.

Evaluate in the loop

Count trials, report intervals, break results down, watch the failures.

Safety sits outside the model

Hard limits below the policy and a human above it.

CREDITS

Image credits and sources

where the pictures and numbers in this deck come from

Robot camera frames and action curves: the LIBERO benchmark (Liu et al., 2023), via the lerobot/libero_10 dataset, Apache-2.0.

Robot photos (Wikimedia Commons, cropped): Unitree G1 by Sayanesy, CC0; Franka by Ims, CC BY-SA 4.0; Flexiv Rizon by MichelleACipriano, CC BY-SA 4.0; MAiRA by Anna Schneider2021, CC BY-SA 4.0.

Toy-model figures (diffusion and flow samples, feature collapse, the simulator gap, domain randomisation, the CLIP matrix and zero-shot examples, number formats) are re-rendered from my own technical videos; the experiments were run for those videos.

π_0 architecture and timing: Black et al., π_0: A Vision-Language-Action Flow Model for General Robot Control, Physical Intelligence, 2024.

World-action model wiring: DreamZero, World Action Models are Zero-shot Policies, NVIDIA GEAR.

Stock photos inside the CLIP and I-JEPA figures: Pexels.

Training recipe: Andrej Karpathy, A Recipe for Training Neural Networks, 2019.

Charts marked illustrative show the typical shape of a curve, not measured data.